The Critical Path: Companion Guide

GPU Performance Engineering

Study Guide

From PMPP to Frontier Inference Kernels

Amiya Diwan

April 29, 2026

North Star Metric

Tokens per dollar.

A kernel earns its place if it raises tokens per second per GPU on the workloads this curriculum targets: long-context decode, attention serving, quantized GEMM, MoE dispatch. The same kernel runs in production serving, RL rollouts, and evaluation. Cheaper inference pays in all three.

Principles

Four tenets shape every exercise and optimization decision in this guide.

1. Follow the problem, not the silo.

The hardest problems cross the boundary between systems engineering, ML research, economics, and hardware design. A slow training run might be a memory-bandwidth bottleneck or a policy gradient variance problem, and you cannot diagnose it from only one side.

2. Prioritize ruthlessly and test cheaply.

Before building infrastructure around an idea, find the riskiest assumption, the one whose failure would make the rest irrelevant, and test it at the smallest informative scale. A 1M-parameter model on a single GPU can hit the same memory wall as a 70B model on a cluster, provided the experiment isolates that specific effect.

3. Cultivate taste.

Develop your own understanding alongside using AI. There are no shortcuts, but the reward is that AI becomes your tool, not your crutch. The deliberate practice loop: **predict** an outcome from principled reasoning, **validate** outputs, **explain** the results, and **prioritize** the next loop with the highest probability of success.

4. Don't overfit to the stack.

Master current tools deeply, but extract the transferable reasoning about *why* systems behave the way they do. What counts as a valuable skill shifts year to year. An engineer who understands the principles on one platform can transfer to another in days, not months.

What "done" means for each problem:

1. Write a naive version and explain its bottleneck.
2. Write an optimized version and *predict* the speedup before measuring.
3. Explain the gap between predicted and actual.
4. Name the transferable principle, not just the CUDA syntax.

Contents

1 Phase 1: Hello World **3**

2 Phase 2: The Architecture Turn **4**

3 Phase 3: Parallel Patterns **6**

4 Phase 4: First Cross-Entropy Pass **7**

5 Phase 5: Bridge to Inference **8**

6 Phase 6: Critical Kernel Implementations **10**

7 Phase 7: KernelBench Challenge **12**

8 Phase 8: Capstone: KernelLLM Fine-Tune **13**

9 Phase 9: Portfolio and Circular Read **13**

10 Timeline **14**

11 References **14**

About this revision (April 2026)

Why this curriculum exists. The research goal is reducing the cost of the experimental loop that produces better models. The April 2026 literature review identified three workloads where kernel throughput is a load-bearing cost: RLVF rollout generation (autoregressive decode is 60–80% of RLVF wall time), adaptive-depth training in the Mixture-of-Recursions family, and long-context experiments. The exercises below target the hand-written boundary where `torch.compile` and CUTLASS stop paying off. The workloads at the frontier will change; the need for hand-written kernels at that boundary will not.

Companion codebase. kernel-forge (`~/Desktop/Karpathy/kernel-forge/`) provides the build system, runner, profiling harness, and stub kernels. You write the kernels; the repo measures, verifies, and plots them. Design spec: `docs/specs/2026-04-19-kernel-forge-design.md`. Paper catalog with PDFs: `~/Desktop/Karpathy/papers/kernel-forge/INDEX.md`.

Trimmed critical path. An April 2026 literature review confirmed seven exercises are load-bearing for the research goal, in this order:

#	Exercise	Transferable principle	Phase
1	matmul (naive)	Memory-bound bottleneck analysis	2
2	matmul (tiled)	Shared-memory tiling reduces global traffic	2
3	reduction	Tree parallelism, warp shuffle	3
4	cross_entropy (fused)	Kernel fusion eliminates materialization	4/6
5	flash_attention	Online algorithms replace materialization	6
6	quantized_gemm	Bandwidth optimization via reduced precision	6
7	moe_dispatch	Irregular parallelism, gather-compute-scatter	6

Exercises retained from the original guide but *not* on the critical path (histogram, conv2d, prefix-sum, sort, trimul) are labelled **optional**. They teach real concepts but are not prerequisites for the research direction.

Phase 1: Hello World

Goal

Working CUDA toolchain, basic thread indexing. ∼2 hours.

Read PMPP Ch 1 (skim, ∼15 min)

Section 1.1 only: the throughput-vs-latency mental model and the host/device distinction. CPUs optimize for latency on a single thread; GPUs optimize for throughput across thousands.

Read PMPP Ch 2, Sections 2.1–2.5 (∼30 min)

Section	Key Concept
2.1	Data parallelism: independent computations on different data
2.2	CUDA C++ program structure: host code, kernel launches, grids
2.3	Vector addition example: the "Hello World" of CUDA
2.4	Device global memory and data transfer (`cudaMalloc`, `cudaMemcpy`)
2.5	Calling kernel functions (`«<blocks, threads»>`)

Warmup: vector_add (archived)

Toolchain Smoke Test: 30 minutes max

This warmup lives at `archive/pmpp_v2/vectoradd_py/` (completed popcorn submission). If you're rebuilding the toolchain on a fresh box, confirm you can launch a one-element-per-thread kernel and read back the result.
Predict: What happens if you launch fewer threads than elements?
If toolchain setup takes more than 30 minutes, your environment is broken. Fix it before touching anything harder.

Read PMPP Ch 3 (∼30 min)

Multidimensional grids and data. Mapping threads to 2D/3D data, row-major linearization, the relationship between `blockIdx`, `threadIdx`, and flat array indices.

Warmup: grayscale (archived)

Lives at `archive/pmpp_v2/grayscale_py/`.

First Predict-Before-Running Exercise

Predict: Does the weighted sum formula ($0.2989R + 0.587G + 0.114B$) affect memory access pattern, or just arithmetic?

Phase 2: The Architecture Turn

Goal

Understand GPU hardware well enough to make informed predictions. You do matmul twice; it is the most important exercise sequence in the plan. 4–5 hours.

Read PMPP Ch 4, Sections 4.1–4.4 (∼30 min)

Section	Key Concept
4.1	Architecture of a modern GPU (SMs, SPs)
4.2	Block scheduling
4.3	Synchronization and transparent scalability
4.4	Warps and SIMD hardware: 32 threads in lockstep

Read PMPP Ch 4, Sections 4.5–4.6 (∼20 min)

Section	Key Concept
4.5	Control divergence: why branches are expensive
4.6	Warp scheduling and latency tolerance: how the GPU hides memory latency

Read PMPP Ch 5, Sections 5.1–5.3 (∼35 min)

Section	Key Concept
5.1	Memory access efficiency
5.2	CUDA memory types (global, shared, constant, registers)
5.3	Tiling for reduced memory traffic

Before coding: Trace the tiled matmul algorithm by hand on a 4×4 example.

`kernels/matmul/1_naive.cuh` (critical path #1)

Implementation target: fill in the body of `sgemm_naive` in `kernels/matmul/1_naive.cuh`. Then:

```
make build
make run EX=matmul K=1                  # your naive SGEMM
make run EX=matmul K=0                  # cuBLAS baseline for the same shape
make profile EX=matmul K=1              # ncu --set full
```

Predict Before Profiling

Predict: What fraction of peak FLOP/s will you achieve? (Expect <5%.)
Compare measured GFLOP/s against `K=0` (cuBLAS). **Explain:** Why is it slow, in terms of memory transactions per FLOP?

Read PMPP Ch 5 remaining + Ch 6, Sections 6.1–6.4 (∼40 min)

Memory coalescing, occupancy, shared memory bank conflicts. You now have a slow kernel. This reading should feel *urgent*: you are diagnosing a real problem.

`kernels/matmul/2_tiled.cuh` (critical path #2)

```
make run EX=matmul K=2                  # your tiled SGEMM
make bench EX=matmul                    # sweep K=0,1,2 across sizes
make plot EX=matmul                     # kernel_0 vs kernel_1 vs kernel_2 chart
```

The Most Important Exercise in the Plan

Predict the speedup before measuring.
Explain the gap between predicted and actual.
If you understand *why* your tiled kernel is faster, not just that it is, you have internalized the core of GPU performance engineering.

Read PMPP Ch 6, Sections 6.5–6.9 (∼30 min)

Section	Key Concept
6.5	Thread coarsening
6.6	Loop unrolling
6.7	Double buffering
6.8	A checklist of optimizations
6.9	Optimization strategy

Phase 3: Parallel Patterns

Goal

Implement each fundamental parallel pattern. Critical path only: 1–2 hours (reduction). Full phase with all optional exercises: 6–8 hours.

On the critical path: `reduction` (kernel-forge exercise #3). Everything else in this phase is optional reading.

Reduction: Tree-Based Parallel Algorithms (critical path #3)

Read: Ch 10, Sections 10.1–10.5 (∼30 min): reduction trees, simple kernel, control divergence, memory access divergence.
Code: Fill `kernels/reduction/1_divergent.cuh` — the textbook tree with `if (tid % (2*stride) == 0)` guards. Measure.
Read: Ch 10, Sections 10.6–10.11 (25 min): reducing global memory accesses, synchronization overhead, thread coarsening.
Code: Fill `kernels/reduction/2_coalesced.cuh` — contiguous active lanes, then `__shfl_down_sync` for the final warp. Compare.

```
make run EX=reduction K=1              # divergent baseline
make run EX=reduction K=2              # coalesced + warp shuffle
make profile EX=reduction K=2          # look at warp execution efficiency
```

Transferable principle: Reduction is a primitive in every ML framework (loss functions, norms, softmax denominators). The tree structure generalizes to any associative operation. The warp-shuffle pattern returns in the softmax step of Phase 6 fused CE and FlashAttention.

Histogram: Contention via Privatization *(optional)*

Read: Ch 9, Sections 9.1–9.4 (∼25 min): atomics, basic histogram kernel, latency and throughput of atomic operations, privatization.
Code: Write naive global atomics first, then privatize to shared memory. Do both versions.
Read: Ch 9, Sections 9.5–9.6 (15 min): thread coarsening, thread-level privatization.

Transferable principle: Contention management via privatization. The same pattern appears in gradient accumulation, token counting, and anywhere concurrent writes collide.

Convolution: Halo Exchange *(optional)*

Read: Ch 7, Sections 7.1–7.4 (∼25 min): background, basic kernel, memory bandwidth considerations, constant memory and caching.
Code: Basic parallel convolution.
Read: Ch 7, Sections 7.5–7.6 (20 min): tiled convolution with halo cells, using caches for halo cells.

Transferable principle: Halo exchange. The same pattern appears in distributed training, stencil computations, and fluid dynamics.

Scan: Work-Efficiency vs Span *(optional)*

Read: Ch 11, Sections 11.1–11.2 (∼25 min): scan background, Kogge-Stone algorithm.
Code: Implement Kogge-Stone.
Read: Ch 11 remaining (20 min): work efficiency, Brent-Kung, segmented scan.
Predict: Kogge-Stone vs Brent-Kung: which wins at $N = 1024$ vs $N = 10^6$?

Transferable principle: The work-efficiency vs span tradeoff is the fundamental tension in parallel algorithm design.

Sort: Composing Primitives *(optional)*

Read: Ch 13 (merge) + Ch 14, Sections 14.1–14.5 (∼40 min): merge algorithm, co-rank function, radix sort, parallel merge sort.
Code: Compose scan + merge into a sort.
Read: Ch 14, Sections 14.6–14.8 (15 min): thread coarsening to improve memory coalescing, other methods.
Predict: At what N does your GPU sort beat `torch.sort`?

Transferable principle: Composing parallel primitives. Sort = scan + merge + scatter.

Phase 4: First Cross-Entropy Pass

Goal

Compose reduction + element-wise into a single-row kernel. Critical path only: 2–3 hours. With optional trimul: 3–4 hours.

On the critical path: `cross_entropy (naive)` — kernel-forge exercise #4, variant 1. The fused variant is revisited in Phase 6.

`kernels/cross_entropy/1_naive.cuh` (critical path #4, warmup)

Cross-entropy fuses softmax (reduction), log, and element-wise operations into one pass:

$$L = -\sum_i y_i \log\left(\frac{e^{x_i}}{\sum_j e^{x_j}}\right)$$

Fill in `cross_entropy_naive`. Three passes per row: max, exp-sum, gather-target. This is the "before" against which the fused version in Phase 6 is measured.

```
make run EX=cross_entropy K=1
make profile EX=cross_entropy K=1      # look at HBM reads per row
```

Predict: How many times is each logit read from HBM in the naive version?

`trimul` (BioML) *(optional)*

Triangle matrix multiplication. Can you modify your tiled matmul for triangular structure? Tests principle 1: follow the problem, not the silo. Lives in `archive/princeton/` if you want the original popcorn problem framing.

Phase 5: Bridge to Inference

Goal

Connect PMPP hardware knowledge to real inference workloads. Set up profiling infrastructure. ∼half day.

Set up cloud GPU

`ncu` needs direct access to GPU hardware counters. Virtualized instances from the major hyperscalers (GCP/AWS/Azure) typically block these counters at the hypervisor level, which causes `ERR_NVGPUCTRPERM` on every profile. Bare-metal or privileged-container providers work:

Provider	Access	A100	Profiling
Lambda Labs	Bare-metal	$1.29/hr	works out of the box
Vast.ai	Docker (privileged)	$0.70–1.10/hr	needs `-privileged`
RunPod	Docker (Secure Cloud)	$0.80–1.20/hr	use Secure Cloud tier

Validation step (run before a long profiling session):

```
make build
make run EX=matmul K=1                 # kernel runs
ncu --set full ./build/matmul 1        # counters accessible
```

If `ncu` returns `ERR_NVGPUCTRPERM`, try a different instance. Set `CUDA_COMPUTE_CAPABILITY` in `CMakeLists.txt` to match the GPU (L4=89, A100=80, H100=90).

Autoregressive Decode Arithmetic Intensity

Think First (no computer)

Compute the arithmetic intensity for single-head attention decode with $d = 128$, 32 heads, context length $S = 2048$.
Per head: $4dS$ FLOPs, $\approx 4dS$ bytes read.

$$I = \frac{\text{FLOPs}}{\text{bytes}} = \frac{4dS}{4dS} \approx 1 \text{ FLOP/byte}$$

On a chip with 1000 TFLOP/s peak compute and 3 TB/s bandwidth, the ridge point is ≈ 333 FLOP/byte. At intensity 1:

$$\text{Throughput} = I \times \text{bandwidth} = 1 \times 3 \times 10^{12} = 3 \text{ TFLOP/s} \quad (0.3\% \text{ of peak})$$

Key insight: Autoregressive decode leaves the arithmetic units idle >99% of the time, waiting for data. Faster compute does not help. Only bandwidth or data movement reduction helps.

FlashAttention Roofline Profiling

Profile FlashAttention decode latency across batch sizes 1, 8, 32, 64. For each batch size, compute:

- Achieved HBM bandwidth (bytes loaded / latency)
- Arithmetic intensity (FLOPs / bytes loaded)
- Attainable TFLOP/s from the roofline model

Plot all four points on the roofline diagram. Identify the crossover batch size where decode transitions from memory-bound to compute-bound.

Read rl-at-scale Ch 2, Move 2 (p. 18–21)

Companion read: Kiely, *Inference Engineering* (Baseten, 2026), §2.4 derives the ops:byte ratio for an H100 (989 TFLOP/s, 3.35 TB/s, ridge ≈ 295) and arrives at 62 ops/byte for $N = 4096$, $d = 128$ decode attention. Same numbers as the derivation above, framed for inference engineers rather than RL researchers; useful as an independent check. Ch 3 covers GPU hardware from a spec-sheet angle, parallel to PMPP Ch 4.

Three kernel categories dominate RLVR inference wall time:

- **Attention kernels:** Naive attention materializes $O(N^2)$ intermediate scores. FlashAttention tiles Q, K, V into SRAM blocks, uses online softmax to accumulate without materializing the $N \times N$ matrix. HBM traffic drops from $\Theta(N^2 + Nd)$ to $\Theta(Nd)$.
- **Quantized matmuls:** Arithmetic intensity scales with precision. For an $N \times N$ GEMM:

Precision	Bytes/elem	Data loaded	Intensity
FP16	2	$6N^2$ bytes	$N/3$
FP8	1	$3N^2$ bytes	$2N/3$
FP4	0.5	$1.5N^2$ bytes	$4N/3$

  For decode (effective $N \approx$ batch size $\times\, d$, typically $\ll 4096$), all precisions remain memory-bound.
- **MoE routing:** Gather-compute-scatter with irregular batch sizes per expert, load imbalance,

and dispatch overhead.

Phase 6: Critical Kernel Implementations

Goal

Build kernels at the hand-written boundary, where `torch.compile` and CUTLASS leave performance on the table. All simplified: algorithmic insight over micro-optimization. 4–6 weeks.

North Star Metric

From Phase 6 on, correctness is the floor. A kernel earns its place by moving tokens per dollar on one of the target workloads: attention decode, fused CE backward, INT4 GEMM, or MoE dispatch.

Each kernel below targets a specific inference bottleneck. Kiely Ch 5 covers these techniques at the systems level (quantization, kernel fusion, parallelism, KV caching, disaggregation); read it for context on *why* these kernels matter inside vLLM, SGLang, and TensorRT-LLM, not for the kernel implementations themselves.

Technique	Bottleneck addressed	Exercise
Online-softmax fusion	materializing the vocab-sized logit tensor	`cross_entropy` (#4)
Tiled streaming attention	HBM traffic on the $N \times N$ score matrix	`flash_attention` (#5)
Weight-only INT4 with fused dequant	decode bandwidth floor	`quantized_gemm` (#6)
Gather–compute–scatter dispatch	irregular per-expert work, launch overhead	`moe_dispatch` (#7)

`kernels/cross_entropy/2_fused.cuh` (critical path #4, fused)

Fill in `cross_entropy_fused` using the online-softmax recurrence: running max m and denominator d, updated in a single pass over the logit row. Same recurrence you will use for FlashAttention next.

```
make run EX=cross_entropy K=2          # your fused version
make bench EX=cross_entropy            # K=1 vs K=2 bandwidth curve
```

Predict Before Fusing

Predict: How many DRAM round-trips does the naive (K=1) version require? How many does the fused (K=2) version need? What is the ratio?
Measure the bandwidth savings. The `gbps` column in the benchmark CSV is the question.

Frontier: The Liger-Kernel Fused-Linear-Cross-Entropy (FLCE, Hsu et al., arXiv:2410.10989) extends this by chunking the logit projection, softmax, CE, and backward into one pass without materializing the vocab-sized logit tensor (∼60% memory reduction). `torch.compile` cannot discover this pattern — it must be hand-written. Read after your fused kernel works.

Transferable principle: Kernel fusion eliminates intermediate materialization. Any pipeline of element-wise and reduction operations is a fusion candidate.

`kernels/flash_attention/*.cuh` (critical path #5, ∼2 weeks)

Target: FlashAttention-2 forward (Dao, arXiv:2307.08691). Single-head, FP32, no masking, no dropout, forward pass only.

Core algorithm: Online softmax (running max + denominator), tiled QKV streaming through SRAM, accumulate output without materializing the $N \times N$ attention matrix.

Not in scope: Tensor cores (`mma.sync`), async copy (`cp.async`), multi-stage pipelines, register pressure optimization.

HBM traffic drops from $\Theta(N^2 + Nd)$ to $\Theta(Nd)$. This is the same tiling principle from PMPP Ch 5 applied to attention.

Path:

1. Fill `1_naive_attention.cuh` — materializes the $N \times N$ score matrix as a correctness baseline.
2. Fill `2_flash_forward.cuh` — the online-softmax tiled version.
3. `make bench EX=flash_attention` — watch the GFLOP/s gap close to naive at large N as memory traffic stops dominating.

Predict Before Implementing

Predict: For $N = 8192$ and $d = 128$, what is the ratio of naive attention HBM traffic to FlashAttention HBM traffic?
After implementing: Does your measured bandwidth match the roofline prediction from Phase 5?

Frontier (read after FA-2 works):

- **FlashAttention-3** (Shah et al., arXiv:2407.08608) — Hopper-specific: warp specialization, async MMA, FP8.
- **FlashAttention-4** (Zadouri et al., arXiv:2603.05451) — Blackwell: hardware-friendly producer-consumer pipelines.

All three extend the same algorithmic core; FA-2 is prerequisite. The same online-softmax state also underpins Ring Attention and PagedAttention.

Transferable principle: Online algorithms (streaming computation with bounded memory) replace materialization wherever intermediate tensors exceed SRAM capacity.

`kernels/quantized_gemm/*.cuh` (critical path #6, ∼1–2 weeks)

Target: INT4 weight-only GEMM with fused dequantization (Marlin, Frantar et al., arXiv:2408.11743 is the production reference).

Scope: INT4 weights dequantized to FP32 on-the-fly, fused with matmul. Activations stay FP32. Load INT4 weights (0.5 bytes/element), dequantize in registers, accumulate in FP32. Never materialize a dequantized weight tile.

For decode at batch=1, the weight-matrix load from DRAM sets a hard floor on latency. INT4 quarters the bytes loaded, quartering the minimum latency. This is a bandwidth optimization, not a compute optimization.

Predict Before Dequantizing

Predict: For a 7B model at H100's 3.35 TB/s bandwidth, what is the minimum decode latency in FP16 vs INT4? (Hint: 14 GB/3350 GB/s vs 3.5 GB/3350 GB/s.)

Frontier (read after your INT4 kernel works):

- **FireQ** (arXiv:2505.20839) — INT4-weight, FP8-activation fused kernel.
- **LiquidGEMM** (arXiv:2509.01229) — W4A8 kernel pipelining.
- **FP6-LLM** (Xia et al., arXiv:2401.14112) — FP6 algorithm–system co-design.
- **RaZeR** (arXiv:2501.04052) — NVFP4 quantization on Blackwell with native hardware support.
- **DeepGEMM** (DeepSeek) — unified FP8/FP4/BF16 kernel library, production-grade.

Transferable principle: For memory-bound kernels, reducing precision is a bandwidth optimization. The compute (FP32 accumulation) stays the same; only the bytes moved change.

`kernels/moe_dispatch/*.cuh` (critical path #7, ∼2 weeks)

Target: Top-1 gather–compute–scatter across E experts, staged into three kernels (count, build permutation, per-expert GEMM, unpermute).

Context: MoE forward passes underperform their theoretical arithmetic intensity due to dispatch overhead, load imbalance, and kernel-launch latency. Rollout-time attention and MoE routing are the two dominant costs in contemporary RLVF training.

Frontier (read after your staged version works):

- **MegaBlocks** (Gale et al., arXiv:2211.15841) — reformulates MoE as block-sparse GEMM so the expert dispatch collapses into a single grouped-GEMM launch.
- **MegaScale-MoE** (arXiv:2505.11432) — hybrid expert parallelism with warp-group-level dispatch and communication overlap.
- **AdaFuse** (arXiv:2603.11873) — adaptive fused switching kernels that eliminate separate routing/gating/FFN launches.

Transferable principle: Irregular parallelism. When work per thread varies dynamically, uniform resource allocation wastes compute. The same challenge appears in graph algorithms, sparse matrix operations, and variable-length sequence batching.

Phase 7: KernelBench Challenge

Goal

Validate kernel skills against a standardized benchmark. Prepare for capstone. 2–3 weeks.

Stanford's KernelBench: 250 tasks across three difficulty levels.

Level	Description	Target
L1	Simple operations: validate skills	Solve most
L2	Standard patterns: matmul variants, attention, norms	Match LLM baselines
L3	Complex fusions: multi-kernel pipelines	Attempt, learn from failures

Meta-goal: Build intuition for what makes kernel generation hard. What patterns do LLMs

struggle with? This directly informs the capstone.

Phase 8: Capstone: KernelLLM Fine-Tune

Goal

Apply RL to improve LLM kernel generation on a narrow domain. 6–8 weeks.

Base model: Meta's KernelLLM-8B (open-weight, HuggingFace).
Target language: Triton (KernelLLM's native target; learned here in context).
Domain: Narrow: attention kernels or reduction/scan patterns (pick one).
RL algorithm: GRPO with execution-time reward.
Training loop: Generate kernel → compile → execute → measure → reward → update.

Hand-written CUDA vs Triton vs torch.compile. After Phase 6 you'll have the CUDA intuition to judge each tool's boundary:

Tool	Handles well
`torch.compile`/Inductor	pointwise fusions, simple norms, matmul epilogues
Triton	fused ops where discovery matters more than peak (Liger-Kernel, DR-Triton)
Hand-written CUDA	FLCE, FlashAttention fwd+bwd, sub-byte quant fusions, comm-overlap, activation recompute inside fused kernels, peak attention/GEMM
CUTLASS/CuTe	production GEMM optimization
ThunderKittens	FA-style tiled kernels at a higher abstraction than raw CUDA

Existing work (CUDA-L1, DRTriton, Dr. Kernel) trains on broad benchmarks. A narrow-domain fine-tune with carefully designed reward shaping on a specific kernel family is a focused contribution. Phase 6 experience building kernels by hand provides unique insight into reward design: you know what makes a kernel *fast* rather than just correct. Reimplementing a Phase 6 kernel (e.g., fused CE) in Triton before training is a natural bridge exercise.

Phase 9: Portfolio and Circular Read

Goal

Demonstrate transferable understanding and end-to-end systems thinking. 1–2 weeks.

Cross-Platform Attention Optimization: Map your FlashAttention implementation from CUDA to TPU/Pallas. Focus on the conceptual mapping:

GPU (CUDA)	TPU (Pallas)
Shared memory tiling	VMEM prefetching
`__syncthreads()`	Implicit tile synchronization
Double buffering in shared memory	`Buffered(buffer_count=2)` DMA

Tests principle 4: if you can only write it in CUDA, you learned syntax, not concepts.

Portfolio artifacts:

- GitHub repository with all kernel implementations
- KernelBench results and analysis
- KernelLLM fine-tune results, model weights, training curves
- Written analysis connecting hardware reasoning to optimization decisions

Circular read: Reread PMPP Ch 4–5 and rl-at-scale Ch 2. After building these kernels, the same pages read differently.

Timeline

Phase	Critical path	With optional	Cumulative
1: Hello World	2 hrs	2 hrs	2 hrs
2: Architecture Turn	4–5 hrs	4–5 hrs	∼7 hrs
3: Parallel Patterns	1–2 hrs	6–8 hrs	∼9–15 hrs
4: First Cross-Entropy	2–3 hrs	3–4 hrs	∼12–19 hrs
5: Bridge to Inference	half day	half day	∼16–23 hrs
6: Critical Kernels	4–6 weeks	4–6 weeks	∼5–7 weeks
7: KernelBench	2–3 weeks	2–3 weeks	∼8–10 weeks
8: Capstone (KernelLLM)	6–8 weeks	6–8 weeks	∼16–18 weeks
9: Portfolio	1–2 weeks	1–2 weeks	∼18–20 weeks

Critical-path only, phases 1–5 fit in a long weekend. Phases 6–9 are the real work. The "with optional" column includes the off-path parallel-pattern exercises (histogram, conv2d, scan, sort, trimul); do them if you want the full PMPP tour, skip them if you want to get to kernel-forge Phase 6 fastest.

References

Full paper catalog (with PDFs) at `~/Desktop/Karpathy/papers/kernel-forge/INDEX.md`. Build-system and profiling-workflow inspiration: Simon Boehm, "How to Optimize a CUDA Matmul Kernel for cuBLAS-like Performance" and siboehm/CUDA-MMM.

Foundations

[1] W.-m. W. Hwu, D. B. Kirk, and I. El Hajj, *Programming Massively Parallel Processors*, 4th ed., Morgan Kaufmann, 2022.
[2] A. Diwan, *The Critical Path: RL at Scale*, 2026.

Attention kernels

[3] T. Dao, "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning," arXiv:2307.08691, 2023.
[4] J. Shah *et al.*, "FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision," arXiv:2407.08608, 2024.
[5] T. Zadouri *et al.*, "FlashAttention-4: Hardware-Friendly Attention with Producer-Consumer Pipelines," arXiv:2603.05451, 2026.

Kernel fusion

[6] P. Hsu *et al.*, "Liger Kernel: Efficient Triton Kernels for LLM Training," arXiv:2410.10989, 2024.
[7] "Deep Kernel Fusion for Transformers," arXiv:2602.11808, 2026.

Quantized GEMM

[8] E. Frantar *et al.*, "Marlin: Mixed-Precision Auto-Regressive Parallel Inference on Large Language Models," arXiv:2408.11743, 2024.
[9] "FireQ: Fast INT4–FP8 Fused Kernel for Quantized LLM Inference," arXiv:2505.20839, 2025.
[10] "LiquidGEMM: W4A8 Kernel Pipelining for Quantized GEMM," arXiv:2509.01229, 2025.
[11] H. Xia *et al.*, "FP6-LLM: Efficiently Serving Large Language Models Through FP6-Centric Algorithm–System Co-Design," arXiv:2401.14112, 2024.
[12] "RaZeR: Redundant-Zero Remapping for NVFP4 Quantization," arXiv:2501.04052, 2025.

MoE dispatch

[13] T. Gale *et al.*, "MegaBlocks: Efficient Sparse Training with Mixture-of-Experts," arXiv:2211.15841, 2022.
[14] "MegaScale-MoE: Hybrid Expert Parallelism with Optimized Communication," arXiv:2505.11432, 2025.
[15] "AdaFuse: Adaptive Fused Switching Kernels for MoE," arXiv:2603.11873, 2026.

Adaptive compute / research context

[16] S. Bae *et al.*, "Mixture-of-Recursions: Learning Dynamic Recursive Depths for Adaptive Token-Level Computation," arXiv:2507.10524, 2025.

Inference systems

[17] P. Kiely, *Inference Engineering*, Baseten Books, 2026.

Capstone (Phases 7–8)

[18] Stanford Scaling Intelligence, "KernelBench," 2025. `https://scalingintelligence.stanford.edu/blogs/kernelbench/`
[19] Meta, "KernelLLM-8B," HuggingFace: `facebook/KernelLLM`, 2025.
[20] "CUDA-L1: Contrastive RL for CUDA Optimization," arXiv:2507.14111, 2025.
[21] "DRTriton: Synthetic Data RL for Triton," arXiv:2603.21465, 2026.
[22] "Dr. Kernel: RL for Triton with Anti-Reward-Hacking," arXiv:2602.05885, 2026.

Tooling (non-paper)

- DeepGEMM — DeepSeek's unified FP8/FP4/BF16 kernel library.
- ThunderKittens — Stanford tile-level attention framework.
- Liger-Kernel — LinkedIn's efficient LLM training kernels (Triton).
- AutoKernel — autonomous kernel optimization framework.